StreamCab: Real-Time Taxi Fare Intelligence with Kafka, Spark, and XGBoost

Bibek Dhungana, Xiaodi Shao, Raivath Mallela
Big Data Application

***Abstract*—This paper presents StreamCab, an end-to-end data engineering and machine learning system for real-time taxi analytics and short-term fare prediction. The pipeline replays historical New York City taxi trips into Apache Kafka, processes and aggregates the stream using Apache Spark Structured Streaming, stores results in PostgreSQL, and serves predictions through a dashboard interface. We describe the architecture, data processing methodology, model training strategy, and performance outcomes, then discuss practical lessons learned about reliability, memory constraints, and deployment trade-offs in single-node environments.**

I. Introduction

Urban transportation data is high-volume, time-sensitive, and geographically heterogeneous. Static batch reports are often insufficient for operational decisions such as where demand is increasing and what pricing behavior to expect in the next few minutes. This project investigates whether a compact local stack can deliver near real-time analytics and prediction from historical taxi records replayed as live events.

The objectives of StreamCab are:

- Build a reproducible streaming pipeline from data ingestion to visualization.
- Produce interpretable zone-level aggregate metrics over short windows.
- Train and serve a practical fare predictor under memory constraints.
- Evaluate model quality against a baseline and report engineering trade-offs.

II. Related Work and Background

Streaming architectures commonly use Kafka for event transport, Spark Structured Streaming for stateful transformations, and SQL databases for serving curated outputs [2], [3]. Gradient-boosted trees, especially XGBoost, are widely used for tabular prediction due to strong accuracy-speed trade-offs [4].

Compared to distributed cloud deployments, this project emphasizes local reproducibility and careful memory-aware design. The training pipeline uses batched parquet processing and continued model updates to support large datasets on limited RAM.

III. System Architecture and Design

A. Architecture Overview

The system has six services: PostgreSQL, Kafka (+ producer), Spark/predictor/dashboard app container, consumer, one-shot trainer, and Kafka Control Center.

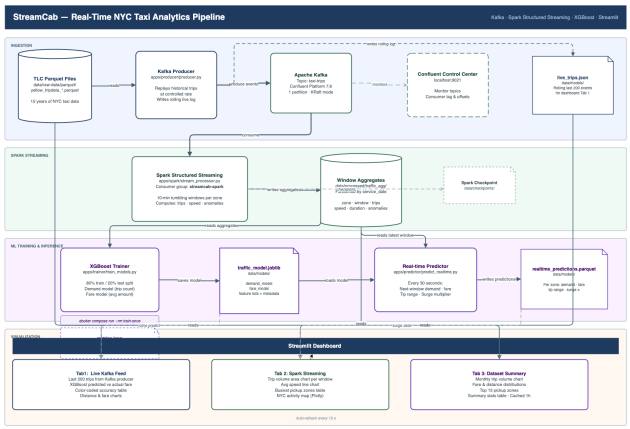

Fig. 1. StreamCab end-to-end architecture.

B. Component Responsibilities

- **Producer:** Streams normalized trip events from parquet in replay loops.
- **Kafka:** Buffers and distributes trip events to downstream consumers.
- **Spark Structured Streaming:** Applies cleaning rules and windowed zone aggregations.
- **PostgreSQL:** Stores live trips, traffic aggregates, and prediction outputs.
- **Trainer:** Builds XGBoost fare model from historical aggregates.
- **Predictor + Dashboard:** Performs periodic inference and displays live analytics.

IV. Data Collection and Processing

A. Data Source

Primary data comes from publicly available NYC TLC yellow taxi trip records (monthly parquet files).

Data source link: NYC TLC Trip Record Data [1]

B. Dataset Profile Used in This Project

The data download utility supports monthly yellow taxi parquet files from 2010 to 2025, with a default pull range of `2022-01` through `2024-12`. For the quick-start workflow documented in the repository, a smaller slice (for example `2022-01` to `2022-03`) is used for rapid iteration and reproducible local runs.

At ingestion time, the pipeline reads only required columns (timestamps, distance, fare/amount, and zone/location fields) and applies consistency filters before aggregation. Rows are retained when duration, distance, speed, and amount fall within physically reasonable ranges, reducing outlier influence and malformed-record noise. For files that do not include pickup zone IDs, pickup latitude/longitude is snapped to nearest TLC zone centroids using a BallTree lookup.

C. Collection Procedure

1) Download selected monthly parquet files.
2) Store files under project raw-data directory.
3) Replay rows as pseudo-live events through Kafka.

D. Schema and Metadata

Raw/event schema (core fields):

- pickup_datetime, dropoff_datetime
- passenger_count, trip_distance
- pu_location_id, do_location_id
- total_amount, emitted_at

Windowed aggregate schema:

- window_start/window_end, pu_location_id, trip_count
- avg_speed_mph, avg_duration_min, avg_trip_distance
- avg_total_amount, anomaly_count

E. Processing Method (Inputs and Outputs)

Inputs are raw parquet trip files and zone centroid metadata. Processing includes type normalization, quality filters, temporal feature extraction, and sliding-window aggregation. Outputs include:

- Streaming aggregate table for current operational state.
- Trained fare model artifact and evaluation metrics.
- Prediction records consumed by dashboard visualizations.

F. Data Quality Rules and Rationale

Before aggregation, event records are filtered to enforce physically plausible trips and to reduce noise from malformed values. The filtering policy is intentionally conservative so that downstream model updates are based on stable patterns rather than outliers.

These constraints reflect common transportation-data cleaning practices: removing implausibly short/long trips, extreme speed artifacts from timestamp errors, and negative or unrealistically large monetary values. In local experiments, this step materially reduced aggregate volatility and improved model training stability.

G. Feature Engineering for Streaming Inference

After filtering, the stream is transformed into zone-window aggregates using fixed 10-minute windows. For each pickup zone and window, the pipeline computes trip count and additive sums (distance, duration, speed, fare), then derives means used by the predictor. Time attributes (hour, day-of-week, weekend flag) are extracted from `window_start`.

This design has two advantages. First, additive aggregation is memory-efficient and naturally composable across micro-batches. Second, zone-window features are interpretable for operations teams: predicted fare changes can be associated with observable shifts in local demand, travel speed, and trip duration.

V. Model Training and Inference Strategy

A. Baseline and Features

The baseline predicts mean fare by pickup zone and hour. The XGBoost model uses zone ID, hour/day/weekend flags, average distance, duration, speed, and trip count.

B. Model Configuration Details

The target variable is `avg_total_amount`. Feature inputs are:

- **Temporal/context:** `pu_location_id`, `hour`, `day_of_week`, `is_weekend`
- **Traffic aggregates:** `avg_trip_distance`, `avg_duration_min`, `avg_speed_mph`, `trip_count`

The XGBoost regressor uses the following hyperparameters:

- `max_depth=5`
- `learning_rate=0.05`
- `subsample=0.9`
- `colsample_bytree=0.9`
- `tree_method=hist`
- `random_state=42`

Training uses a temporal 80/20 train-test split, and the training partition is further divided into core-train plus validation for continued batch-wise boosting.

C. Memory-Aware Training

To support large datasets on limited memory, the trainer:

- reads parquet in bounded batches,
- computes additive aggregate statistics,
- finalizes means after grouped reductions,
- applies continued training across chronological batches (default `250,000` rows/batch),
- selects the best validation checkpoint for final artifact.

D. Objective and Evaluation Metrics

The predictor is trained for squared-error regression and evaluated with both absolute and relative error metrics. Let y_i be ground-truth average fare and $\hat{y}_i$ be the model estimate on sample i.

$$\mathrm{MAE} = \frac{1}{n}\sum_{i=1}^{n} |y_i - \hat{y}_i| \tag{1}$$

$$\mathrm{MAPE} = \frac{100}{n}\sum_{i=1}^{n} \left|\frac{y_i - \hat{y}_i}{y_i}\right| \tag{2}$$

MAE is robust for business interpretation in dollar units, while MAPE gives a scale-normalized view that is useful when zone-level fare ranges differ. Reporting both prevents over-reliance on a single error lens.

Field	Stage	Description
`pickup_datetime`	raw event	Source pickup timestamp used for window assignment
`dropoff_datetime`	raw event	Source dropoff timestamp used for duration computation
`trip_distance`	raw event	Per-trip distance input to quality checks and aggregates
`total_amount`	raw event	Monetary target proxy (or fallback from fare amount)
`pu_location_id`	raw/derived	Pickup zone ID from source or centroid snap for legacy rows
`window_start`, `window_end`	aggregate	10-minute temporal boundaries for grouped statistics
`trip_count`	aggregate	Number of valid trips observed in zone-window
`avg_trip_distance`	feature	Mean distance per zone-window from additive reductions
`avg_duration_min`	feature	Mean trip duration (minutes) per zone-window
`avg_speed_mph`	feature	Mean derived travel speed used as congestion proxy
`avg_total_amount`	target	Supervision target for zone-window fare prediction
`hour`, `day_of_week`, `is_weekend`	feature	Calendar context features derived from `window_start`
`predicted_avg_fare`	serving	Model inference output stored for dashboard consumption

TABLE I
DATA DICTIONARY ACROSS INGESTION, AGGREGATION, AND PREDICTION STAGES.

Rule	Bound / Requirement
Duration	$1 <$ minutes < 180
Trip distance	$0.2 <$ miles < 100
Average speed	$1 <$ mph < 80
Fare / total amount	$0 <$ amount < 500
Pickup/dropoff times	Parseable timestamps
Pickup zone	Zone ID or mappable lat/lon

TABLE II
PRIMARY QUALITY RULES USED BEFORE WINDOW AGGREGATION.

E. Continued Boosting Procedure

Instead of fitting a single monolithic batch, the model is updated chronologically. An initial booster is trained on the first core batch, then expanded with additional trees on subsequent batches. After each update, validation MAE is measured; the checkpoint with the best validation MAE is selected as the final artifact. This procedure reduces peak memory demand and provides a practical early-stopping proxy in constrained environments.

VI. EXPERIMENTAL SETUP AND EVALUATION PROTOCOL

A. Execution Environment

All components were executed in a local, containerized environment orchestrated with Docker Compose. The stack includes Kafka for transport, Spark Structured Streaming for stateful aggregation, PostgreSQL as the serving database, and Python services for training, consumption, inference, and dashboard rendering. This setup prioritizes reproducibility over horizontal scalability, making it suitable for coursework and controlled engineering experiments.

The training and streaming pipelines were tested on commodity hardware with limited memory, which informed several implementation decisions: bounded parquet batch sizes, strict column projection during reads, and additive aggregate statistics rather than full in-memory joins. While this environment does not replicate cloud-scale throughput, it provides realistic pressure for designing robust local-first data systems.

B. Configuration Snapshot

Table III summarizes key runtime and training parameters used in the reference implementation.

Parameter	Default value / role
Window size	10-minute aggregation window
Parquet read batch	500,000 rows per batch
Training batch size	250,000 rows for continued boosting
Validation fraction	10% of train partition (capped by train size)
Initial trees	120 trees for first XGBoost fit
Trees per update	40 trees added per continued batch
Temporal split	80% earlier windows train, 20% recent windows test
Serving sink	PostgreSQL tables for aggregates and predictions

TABLE III
CONFIGURATION SNAPSHOT FOR REPRODUCIBLE LOCAL RUNS.

C. Data Splitting and Validation

Training uses a chronological split to reduce temporal leakage risk. Earlier windows form the primary training set, and a held-out suffix is used for validation and test estimates. This ordering is important because demand and traffic behavior shift over time; random shuffles can overestimate quality when neighboring windows share near-duplicate structure.

We compare two models:

- **Baseline:** pickup-zone and hour mean fare.
- **XGBoost:** gradient-boosted trees over engineered streaming aggregates.

Evaluation metrics include MAE for absolute error and MAPE for relative error, which together capture practical prediction quality for operational dashboards.

D. Runtime and Serving Considerations

Inference is run as a periodic scoring task over the most recent aggregate windows written to PostgreSQL. To keep end-to-end latency stable, predictor queries are intentionally narrow (recent time slice, selected feature columns), and writes are idempotent by timestamp-window keys. This pattern reduces unnecessary read amplification and avoids duplicate prediction rows during restarts.

From an operations perspective, three checks proved useful in continuous runs: (1) stream lag monitoring at Kafka consumer groups, (2) aggregate table freshness checks in SQL, and (3) prediction row count sanity checks per scoring interval.

Signal	Practical objective
Kafka consumer lag	Bounded and non-increasing trend
Aggregate freshness	New zone-windows visible each cycle
Predictor freshness	Prediction table updated on schedule
Dashboard staleness	Latest window rendered without manual refresh loops
Failure recovery time	Service restart without duplicate spikes

TABLE IV
OPERATIONAL SLOS USED FOR LOCAL RELIABILITY CHECKS.

Model	MAE	MAPE (%)
Baseline (zone-hour mean)	1.9891	15.43
XGBoost (test)	0.4851	3.42

TABLE V
EXAMPLE MODEL PERFORMANCE SUMMARY.

These low-cost signals helped detect stalled consumers and schema mismatches early, improving reliability during demos and repeated report experiments.

E. End-to-End Execution Workflow

The practical runbook used for experiments is:

1) Download selected monthly parquet files and verify storage path.
2) Run one-shot training to generate model artifacts (`traffic_model.joblib`, `metrics.json`).
3) Start streaming services (producer, Kafka, Spark pipeline, consumer, predictor).
4) Validate aggregate and prediction table updates in PostgreSQL.
5) Open dashboard for live KPI and map inspection.

This workflow separates expensive offline training from online scoring. As a result, retraining can be scheduled independently from serving availability, reducing runtime coupling and improving demo robustness.

F. Code Availability

The project source code and run instructions are available at: https://github.com/dhunganabibek/StreamCab

G. Suggested Service-Level Objectives

For stable operation, the project tracks lightweight service-level objectives (SLOs) aligned with classroom-scale infrastructure.

VII. RESULTS AND ANALYSIS

A. Quantitative Results

The current run demonstrates a large gap between baseline and boosted-tree performance. The baseline (zone-hour mean) captures coarse temporal and spatial structure, but misses nonlinear interactions between trip count, speed, distance, and duration. XGBoost captures these interactions and reduces both MAE and MAPE.

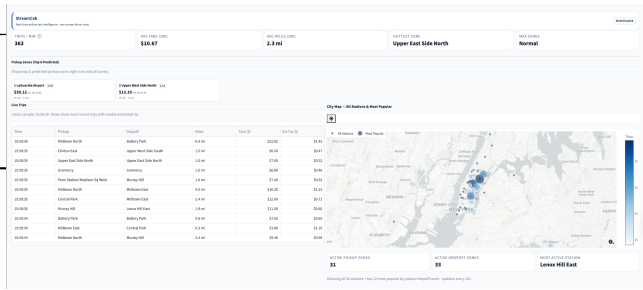

Fig. 2. StreamCab dashboard showing live trip table, zone-level insights, and map-based analytics.

B. Dashboard and Analytics Outputs

Include screenshots or figures for:

- Live trips per minute
- Zone ranking and surge signals
- City map activity visualization

C. Discussion

The pipeline demonstrates that practical near real-time analytics can be achieved in a local setup when ingestion, processing, and training are designed for bounded memory and idempotent writes. SQL-side narrowing for predictor reads and stream checkpointing significantly improve operational stability.

D. Operational Findings and Reliability Lessons

Three implementation choices were especially impactful in day-to-day operation:

- **Strict column projection on parquet reads:** lowered I/O and reduced memory pressure during both aggregation and training.
- **Idempotent database writes by time-window keys:** reduced duplicate rows after container restarts.
- **Bounded checkpointed streaming:** improved restart behavior and reduced recomputation cost.

In practice, the combination of lightweight observability checks (consumer lag, aggregate freshness, prediction-row sanity) was sufficient to catch most runtime issues before they affected dashboard interpretation.

E. Error Analysis by Scenario

The largest residual errors were typically observed during abrupt demand-shift windows where historical averages become poor priors. Examples include sudden surges, sparse late-night zones, and transitions between weekday and weekend travel patterns. While aggregate features are stable, they can smooth away micro-context such as weather, event schedules, and incident-driven rerouting.

This suggests two practical extensions: (1) incorporate exogenous signals (weather/events), and (2) support uncertainty-aware outputs (prediction intervals or quantile models) so the dashboard can distinguish confident estimates from high-variance windows.

F. Limitations and Threats to Validity

Although the quantitative results are strong, several limitations affect generalization and operational confidence. First, the stream is a replay of historical data rather than a true live feed, so observed latency and throughput may differ from production systems with bursty arrival patterns, schema drift, and upstream outages. Second, zone-level aggregates improve model stability but can hide fine-grained effects such as street-level congestion, event-driven anomalies, and rare pickup/dropoff combinations.

The evaluation window is also constrained by selected months and filtering rules. If market conditions shift (fuel prices, policy changes, seasonality, or weather shocks), model error can increase without immediate visibility. Future iterations should include explicit drift indicators (feature distribution shift, residual monitoring, and alarm thresholds) to ensure model performance remains within acceptable bounds. Finally, local single-node deployment simplifies reproducibility, but it does not fully represent fault domains and scaling behavior in distributed environments.

G. Reproducibility and Operational Notes

To support reproducibility, all services are containerized and orchestrated through Docker Compose with deterministic startup dependencies. The project separates concerns across producer, streaming processor, consumer, trainer, and dashboard services. This isolation reduces coupling and makes it easier to troubleshoot failures by layer. The end-to-end runbook consists of: (1) downloading raw TLC parquet data, (2) launching services, (3) running one-shot training, (4) starting replay/streaming, and (5) validating outputs in PostgreSQL and the dashboard.

Operationally, two implementation choices were especially important. The first is checkpointed stream processing with bounded windows, which limits duplicate writes and supports restart safety. The second is memory-aware training via batched parquet reads and incremental model updates, which enables full-dataset learning on commodity hardware. For teams extending this project, we recommend adding lightweight data quality contracts at ingest time (null/negative checks, timestamp sanity, and schema assertions) and documenting expected service-level objectives for latency and prediction freshness.

The architecture can be extended incrementally: move stateful processing to a managed cluster, introduce model registry/versioning, and add automated backtesting before promoting new models. These steps preserve the current educational value while improving production readiness.

VIII. Deployment Trade-Offs and Responsible Use

A. Scalability Roadmap

The local-first design was intentional for rapid iteration and reproducibility, but scaling to larger traffic volumes requires architectural adjustments. A practical roadmap is phased: first, externalize state and monitoring; second, separate compute roles; third, harden model lifecycle.

In phase one, the same logical pipeline can run with managed Kafka and managed Postgres while preserving service boundaries. In phase two, Spark processing and model inference should be decoupled so backlog in one stage does not propagate to the other. In phase three, model artifacts should be versioned with promotion gates (offline validation thresholds and rollback hooks). This progression reduces operational risk while keeping the core feature contract intact.

B. Cost and Complexity Considerations

A key lesson from StreamCab is that reliability improvements are often inexpensive compared to full platform migration. For example, stricter schema checks, idempotent writes, and better observability provide significant stability gains without immediate distributed-cluster overhead.

However, there is a trade-off between implementation simplicity and performance headroom. Small local stacks are easy to reason about and debug, but become brittle under sustained throughput spikes. Distributed deployments improve elasticity but introduce non-trivial complexity in resource tuning, security policy, and incident response. In educational and prototype settings, a staged migration strategy is usually preferable to an immediate all-in rewrite.

C. Data Privacy and Ethical Constraints

Taxi trip records can reveal sensitive mobility patterns at fine spatial and temporal granularity. Even when identifiers are absent, repeated trajectories may indirectly expose behavioral signals. For this reason, outputs should prioritize aggregate reporting over row-level redistribution, and access to raw tables should follow least-privilege principles.

Model predictions should also be interpreted as decision support rather than ground truth. Overconfident use in operational dispatch or pricing may amplify existing geographic inequities if data coverage differs by zone. Recommended safeguards include confidence annotation, periodic bias checks across zones/time windows, and human review for high-impact policy decisions.

These constraints do not prevent practical use; they define guardrails for responsible deployment. Incorporating governance from the start generally improves long-term trust and reduces rework in later productionization stages.

IX. Conclusion and Key Takeaways

What worked well:

- Modular services and reproducible local orchestration.
- Streaming aggregates as robust prediction features.
- Strong tabular prediction performance with XGBoost.

Main challenges:

- Large data volume vs local memory constraints.
- Reliability semantics across stream and database boundaries.
- Balancing latency, correctness, and simplicity.

Future work:

- Add drift monitoring and automated retraining schedules.
- Extend features with weather/events/geospatial context.
- Compare distributed multi-node deployment variants.

REFERENCES

[1] NYC Taxi & Limousine Commission, *TLC Trip Record Data*, https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page

[2] Apache Kafka Documentation, https://kafka.apache.org/documentation/

[3] Apache Spark Structured Streaming Guide, https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html

[4] T. Chen and C. Guestrin, *XGBoost: A Scalable Tree Boosting System*, KDD 2016.